iChat Pro 软件测试实践第四次实验报告

单元测试与代码评审

> 编写日期：2026-10-13。核对代码基线：本地分支工作区，位于 `4703ff3` 之上的文档与测试修订提交。 本报告记录本轮实际执行的单元测试与代码评审过程。测试数据取自 2026-10-13 本机实际运行结果，评审数据取自仓库 Git 提交历史与 GitHub PR 记录。 当前项目状态见 项目现状；文档用途与归档规则见 文档维护索引。

一、实验基本信息

项目	内容
课程名称	软件测试实践
实验名称	单元测试与代码评审
项目名称	iChat Pro 跨平台安全即时通信系统
小组编号	7
实验日期	2026-10-13
小组成员	叶伟智（121052024140）、许栢境（121052024142）、林弘（121052024144）、庞嘉铭（121052024146）
代码仓库	HeanX/iChat_Pro
本轮测试环境	Windows，Python 3.13.12，Node.js 24.14.1，SQLite（本地）/ PostgreSQL 16 + Redis 7（CI）

二、实验目的

1. 说明本项目单元测试的分层结构、用例规模、设计方法和实际执行结果；
2. 说明代码评审的流程、工具、评审清单和真实评审记录；
3. 用可复核的提交记录说明评审如何发现并修复缺陷，而不是只描述制度；
4. 如实记录当前单元测试与代码评审的不足，不把计划写成成果。

三、单元测试体系

3.1 测试框架与分层

本项目后端使用 Django 自带的 `TestCase` 与 `TransactionTestCase`，不引入额外测试框架；前端 JavaScript 使用 Node.js 内置的测试运行器与自写断言脚本。测试按被测对象的职责分层，而不是按文件平均铺开：

层级	主要测试类	覆盖对象
模型与约束	`EncryptedMessageMode`	字段约束、唯一约束、

层级	主要测试类	覆盖对象
	lTests、GroupMessageModelTests、ConversationModelTests、MessageStatusModelTests、KeyTrustTests	状态取值、信任关系
服务层事务与幂等	PrivateHttpIdempotencyTests、PrivateWsReplayTests、GroupIdempotencyTests、ConcurrentSameIdTests	chat/services/messaging.py 的统一写入路径
HTTP 接口与权限	GroupAPITests、ConversationManagementAPITests、PrivacySecurityAPITests、MessageHistoryPermissionTests、CiphertextIntegrityTests	鉴权、越权、参数边界、密文完整性
WebSocket 实时通信	ChatConsumerTests、PrivateRealtimeMessageTests、GroupRealtimeMessageTests、WebSocketPresenceTests	连接、收发、在线状态、成员变更广播
协议与错误码	ErrorRegistryTests、ApiAuthMiddlewareTests、ProtocolVersionGateTests、MultiConnectionPresenceTests	chat/errors.py 注册表、401 JSON、版本门禁
增量补取	SyncEventRecordingTests、SyncCursorPagingTests、SyncVisibilityParityTests、CursorNonAsciiTests	签名游标、分页、可见性投影、游标编码

层级	主要测试类	覆盖对象
E2EE 前端	`private_chat_e2ee.test.js`、`group_chat_e2ee.test.js`、`public_key_cache.test.js`	加解密流程、公钥版本命中、缓存回退
客户端状态机	`chat_connection.test.js`、`message_accepted.test.js`	重连退避、ACK、晚 ACK 行定位与排序
桌面与通知	`desktop_tray.test.js`、`desktop_notifications.test.js`、`chat_notifications.test.js`、`notification_integration.test.js`	托盘生命周期、通知门控、隐私开关
运维脚本	`ichat_pro.tests`	健康检查与配置

3.2 用例规模（2026-10-13 实测）

后端 Django 测试共 **404** 条测试方法，分布在 **77** 个测试类、14 个测试模块中：

测试模块	用例数
`accounts.tests`	127
`chat.tests.test_core`	80
`chat.tests.test_phase2_backend`	54
`chat.tests.test_phase2_issues`	47
`chat.tests.test_t32_review`	18
`chat.tests.test_integration`	16
`chat.tests.test_sync_api`	14
`chat.tests.test_messaging_service`	11
`chat.tests.test_protocol`	11
`chat.tests.test_llm`	9
`chat.tests.test_private_realtime`	6
`chat.tests.test_group_realtime`	5
`chat.tests.test_conversation_api`	3
`ichat_pro.tests`	3

测试模块	用例数
合计	**404**

前端与桌面测试：

测试集	规模
Node.js `node:test` 用例	62 条（7 个文件）
自写断言脚本	4 个（私聊 E2EE、群聊 E2EE、连接状态机、桌面配置）
Chromium 布局回归	36 项（3 种窗口尺寸 × 明暗主题）
Windows 品牌资源校验	22 条 + 原生 PE 图标帧比对
NSIS 卸载宏隔离断言	24 项

3.3 用例设计方法

单元测试不是只走正常路径，针对不同被测对象使用了对应的设计方法：

等价类划分与边界值。 `require_base64` 的校验用例把输入划为合法 Base64、非法字符、长度不足、长度超限四类；`nonce` 固定 12 字节、`auth_tag` 固定 16 字节，测试分别覆盖恰好合法、少一字节、多一字节。文件大小上限、群成员上限（50）、同步分页上限（200）均取边界值两侧。

状态迁移。 消息状态机 `sending → sent → delivered → read` 用 `chat_connection.test.js` 覆盖合法迁移、非法回退、迟到 ACK 把 `failed` 恢复为 `sent`。撤回测试覆盖首次撤回成功、重复撤回返回 `already_recalled`、超时撤回返回 `recall_timeout`。

错误推测与安全边界。 `test_t32_review.py` 专门收集评审中提出的反例：跨会话复用同一 `client_message_id`、群成员版本变更后重放、停用接收者、文件密钥覆盖非法成员、只带 holder 不带密钥材料等，每条都对应一次真实评审意见。

并发与事务。 `ConcurrentSameIdTests` 使用 `TransactionTestCase` 触发真实的唯一约束竞态，验证内层 savepoint 回滚后外层事务仍可恢复查询，而不是用 mock 假装并发。

生产代码直跑。 晚 ACK 的 12 条 DOM 测试直接调用生产环境的处理函数、邻居分组函数和行移动辅助函数，作用于一个 DOM fixture，而不是重新实现一份逻辑来测自己。

3.4 实际执行结果

2026-10-13 在本机执行的完整结果：

```
python manage.py check
python manage.py makemigrations --check --dry-run
python manage.py test
npm run test:e2ee
```

检查	结果
`manage.py check`	System check identified no issues (0 silenced)
`makemigrations --check --dry-run`	No changes detected（模型与迁移一致）
`manage.py test`	**Ran 404 tests — OK (skipped=2)**
`npm run test:e2ee`	4 个脚本套件全通过 + **62** 条 `node:test` 通过，**0** 失败
`desktop: npm run test:layout`	Sidebar layout: **36/36 browser checks passed**
`desktop: npm run test:branding`	22 条通过 + 原生 PE 资源校验通过
`desktop: npm run test:installer`	NSIS runtime: **24/24 assertions passed**

`skipped=2` 说明：有 2 条用例依赖 PostgreSQL 专属行为，本地 SQLite 下跳过；它们在 CI 的 PostgreSQL 16 job 中实际执行。

CI 侧的双后端证据（基线 run 37630595919）：SQLite 与 PostgreSQL 16 / Redis 7 各执行 404 条 Django 测试，两个 job 均通过，说明本地 SQLite 通过不等于正式数据库通过，两者分开取证。

3.5 一个真实的测试缺陷（本轮发现并修复）

单元测试本身也会出错。本轮在复核测试套件时发现一个测试污染缺陷：

`accounts/tests.py` 的 `test_avatar_crop_accepts_real_image` 会向 `/api/profile/avatar/crop/` 提交一张真实图片。该视图通过 Django 默认存储把文件写入 `settings.MEDIA_ROOT`，而测试没有覆盖 `MEDIA_ROOT`，因此每运行一次测试套件，就往仓库的 **`media/avatars/`** 目录写入一个真实文件。

由于 Django 存储在同名文件已存在时会追加随机后缀，这些文件不会互相覆盖，只会持续堆积。取证结果：

- 目录中累计 **88** 个文件，其中 **82** 个是逐字节相同的 78 字节 PNG；
- 这 82 个文件的 SHA-256 全部为 `cf98c02519d19c006f894f145aba49fa3fd603afa3a891754178ed3316a263ab`；
- 用测试输入复跑一遍视图的完整重编码流程（`convert('RGBA')` + `save(format='PNG', optimize=True)`），输出哈希与这 **82** 个文件完全一致，确认全部是测试产物；
- 数据库 `UserProfile.avatar` 只引用 3 个文件，均不在这 82 个之中。

修复方式：为该测试类增加类级临时 `MEDIA_ROOT`，并在类清理阶段删除临时目录。

```python
@classmethod
def setUpClass(cls):
    cls._media_root = Path(tempfile.mkdtemp(prefix='ichat-test-media-'))
    cls.enterClassContext(override_settings(MEDIA_ROOT=cls._media_root))
    cls.addClassCleanup(shutil.rmtree, cls._media_root, ignore_errors=True)
    super().setUpClass()
```

使用 `Path` 而非 `str`，是因为 `chat/views.py` 中存在 `MEDIA_ROOT / 'uploads' / ...` 的路径拼接用法，覆盖成字符串会破坏该写法。

验证：跑完整套件前后 `media/avatars/` 均为 6 个文件，临时目录无残留。清理时只删除哈希与被测输出逐字节一致的 82 个文件，数据库引用的 3 个头像全部保留并验证仍可解析。

这条缺陷的意义在于：测试用例"通过"不代表测试没有副作用。原先 404 条全绿，但绿色之下每次都在污染工作区。

四、代码评审过程

4.1 评审流程

项目采用 `Issue` → 独立分支 → 本地验证 → `Pull Request` → 他人评审 → 合并 → 验收证据 的流程，`main` 保持可演示、可测试。完整协作规则见 GitHub 协作开发指南。

阶段	工具	落地产物
任务与范围	GitHub Issues	78 项 `P4 Txx` / `P4 Test Txx` 任务卡
分支与提交	Git	`feature/`、`fix/`、`test/`、`docs/` 前缀分支
评审载体	GitHub Pull Request	PR 模板 + 评审意见 + 修改提交
自动检查	GitHub Actions	test / integration / javascript / windows-installer
评审清单	代码规范、版本管理与测试工具说明`.md` §3.6	6 类必查项

4.2 评审清单

仓库规定每次评审至少检查：

1. 是否只实现了 Issue 约定的范围；
2. 是否正确处理权限、异常、空值、重复请求和并发；
3. 是否破坏旧客户端、旧数据或既有聊天功能；
4. 是否新增或更新了必要测试；
5. 是否存在明文密钥、隐私日志、注入、越权或不安全文件访问；

6. 测试证据是否可复现，文档和配置是否同步更新。

PR 模板另有"文档与证据同步"三项强制勾选，明确要求"未运行的检查不得写成通过""CI 全绿不代替原生验收"。

4.3 评审规模（实测统计）

截至 2026-10-13，仓库累计 **311** 次提交，其中：

类别	数量
合并提交（对应 PR）	118
`fix` 类提交	79
`feat` 类提交	58
`docs` 类提交	36
`test` 类提交	15
提交信息含 review 的提交	47
明确标注 "review round N" 的提交	**28**

"review round N" 的提交数是评审强度的直接证据：同一个任务因为评审意见反复修改并重新提交，最多的一轮走了 **10** 轮。这与"提交一次就合并"的做法有本质区别。

4.4 高轮次评审实例

P4 T21（敏感数据存储与日志）—— **8** 轮评审。

提交	该轮评审解决的问题
`f455ca3`	桌面端禁止明文降级、密钥按账号隔离、默认强制 HTTPS、草稿与 AI 历史加密
`f62c980`	KV 迁移/清除/脏标记守卫、字节上限、导入绑定、开发模式优先级
`e5366d0`	KV 层六处修复
`9ac65b2`	重新落地——上一轮修复在提交前丢失
`6ae5212`	hydrate 的 epoch/parse 守卫、持久化反馈、2MB 上限
`c283f58`	迁移时不自删、每个 await 点做 epoch 检查、横幅空值、解码余量
`d1de91e`	按 key 记录持久化失败
`526a79a`	`secureKvClear` 清除该 key 的持久化失败状态

其中 `9ac65b2`（"re-land"）尤其值得记录：上一轮的修复在提交前丢失了，评审发现后重新落地。这说明评审不只是看代码风格，而是逐轮核对上轮意见是否真正进入提交。

P4 T15（**Windows** 打包与卸载）—— **4** 轮评审。

提交	该轮评审解决的问题
`5f8e602`	卸载选项改为显式提示，不再静默删除应用数据
`36e4cde`	移除静默删除，用户数据目录名修正为 `ichat-pro-desktop`，默认按钮改为"否"
`e66fd6e`	同上，修正 YES/NO 分支、先离开 INSTDIR 再 RMDir、限定注册表删除范围
`a2207d7`	强制清理 INSTDIR 残留与陈旧卸载登记
`45a7819`	全 raw NSIS 宏、强制清理尾部语义
`b46dfc1`	修正 YES/NO 分支、离开 INSTDIR、限定注册表删除
`09c7d51`	注册表清理按 UninstallString 位置限定
`be8fd11`	引号感知的登记匹配

P4 T20（连接状态与自动重连）—— **10** 轮评审；**P4 T33**（消息发送状态机）—— **9** 轮评审。T33 的评审轮次包括队列死锁、群组去重恢复、异步测试 runner、时间线有序插入、行级标记、邻居分组重建等，每一轮都对应一个具体的可复现问题。

4.5 评审发现并修复的缺陷

评审产出的不是"意见清单"，而是可复现的缺陷与对应回归用例。以下是已记录闭环的四例：

缺陷 **1**：晚 **ACK** 错行（严重度：高）

- 现象：补取把接收到的消息插到延迟发送的消息之前时，按新数组下标去改旧 DOM 顺序，会替换掉一条无关的行。基线复现为数组 ID `[43,44,45]` 而 DOM 为 `[45,44,45]`。
- 评审发现路径：T33/T20 多轮评审逐轮核对行身份定位方式。
- 修复：ACK 处理改为定位并重标原临时行，用服务端时间更新数组，移动完整行后再按新旧两个位置重建分组；找不到行或后继行时整体重渲染。
- 回归用例：`message_accepted.test.js` 12 条，覆盖尾部/前部/中部、时间戳并列、重复 ACK、选中身份、缺失行/旧行/系统行、AI 容器隔离。

- 真实环境复验：真实桌面断网发送 + 第二浏览器发两条消息 + 延迟 3 秒重发，9.241 秒恢复，数组与 DOM 均为 `[46,47,48,49]`，刷新无重复。

缺陷 **2**：公钥版本被最新缓存版本替代（严重度：高，属加密边界）

- 现象：显式指定公钥版本时，被最新缓存版本替代；密钥轮换后既有加密测试暴露回归。
- 修复：显式版本必须精确命中缓存，无版本离线发送保留最新密钥回退。
- 回归用例：`public_key_cache.test.js` 2 条，覆盖私聊与群聊两个模块。

缺陷 **3**：卸载登记前缀匹配误删（严重度：高，涉及用户数据）

- 现象：卸载登记只按安装目录前缀匹配，可能误删路径相似的另一次安装；全机卸载可能选错用户数据目录。
- 修复：改为引号感知的精确匹配，按 UninstallString 位置限定清理范围。
- 验证：24 项真实 NSIS 宏隔离断言，覆盖路径/参数边界、独立登记、是/否/静默分支和安装模式。

缺陷 **4**：通知预览泄露与布局回归（严重度：中）

- 现象：通知设置未就绪时可泄露消息预览；未知会话未查明静音即通知；内嵌设置开关引起侧栏滚动约 695px 并挤压搜索头部。
- 修复：通知门控按"设置未就绪即关闭"处理，未知会话先读元数据查静音；布局修复重排侧栏 flex 尺寸与聊天顶部定位。
- 验证：通知三套回归接入 CI；Chromium 布局 36 项，同套用例在修复前的基线上失败——这条"基线失败、修复后通过"的记录是回归测试有效性的关键证据。

4.6 评审纪律

仓库明确规定了几条防止评审流于形式的约束：

- 禁止直接向稳定分支提交未经测试的功能；
- 禁止通过删除或跳过失败测试来"修复"流水线；
- 禁止伪造测试记录、覆盖率、AI 输出或验收截图；
- 禁止仅凭 AI 生成结果合并代码，所有结果必须经人工审查和可复现验证；
- 关键安全或 E2EE 改动应由非作者成员复核；
- "代码存在""测试已编写""本地能打开页面"不能当作跨端安装与公网验收通过。

最后一条在本轮任务中再次被验证：测试套件 404 条全绿，但 `media/avatars/` 仍在被持续污染——CI 绿不等于没有问题。

五、当前不足（如实记录）

1. 没有覆盖率数据。`coverage.py` 在工具清单中列为"计划接入"，尚未实际执行，因此本报告不提供语句/分支覆盖率数字。需求文档中 85%/75%/90% 的覆盖率目标是目标值而非实测值。
2. 没有静态检查与安全扫描。Ruff、Bandit、pip-audit/npm audit 均未接入，格式与安全扫描未闭环。
3. CI 未强制阻断合并。GitHub `main` 未配置 `required_status_checks`，rulesets 为空。现有四个 job 会运行并通过，但"测试失败禁止合并"目前是协作约定，不是技术强制。
4. 测试分布不均。`accounts.tests` 单文件 127 条、`test_core.py` 80 条，两个文件占全部用例的一半以上；而 Android 相关测试为 0，因为 Android 工程尚未开始。
5. 缺少评估测试质量的指标。本轮修复的测试污染缺陷说明：用例数量、"全绿"结果都不能说明测试本身是健康的，项目目前没有变异测试、测试副作用检查等手段。
6. 评审依赖单人环境。仓库提交者以单一账号为主，多人评审更多体现在流程要求而非多个独立评审人的记录上。

六、实验结论

1. 后端 404 条、前端 62 条、桌面 36+22+24 项测试在本轮全部实际执行并通过，`check` 与迁移一致性检查无问题。
2. 单元测试按模型、服务层、HTTP、WebSocket、协议、补取、E2EE、客户端状态机、桌面和运维分层设计，使用了等价类、边界值、状态迁移、错误推测和真实并发等不同方法，不是单一模板的重复。
3. 代码评审有真实载体（118 个已合并 PR）、真实强度（28 次 review round，最高 10 轮）和真实产出（可复现缺陷 + 对应回归用例 + 修复后验证）。
4. 本轮发现并修复了一个测试自身的缺陷：头像测试长期向仓库 `media/` 目录写入文件，累计 82 个可证明的测试产物，已通过临时 `MEDIA_ROOT` 隔离修复并清理。
5. 项目的测试体系仍有明显缺口：无覆盖率、无静态扫描、CI 不强制阻断合并、Android 无测试。这些是后续实验需要推进的内容，本报告不将其记为已完成。